

Built, deployed, and reproducible

Yes — and every number on the following pages was produced by running the command named beside it, on this repository. Every claim is paired with the way a judge can reproduce it in under a minute. The final pages set out the scope the system covers today, the direction it extends in, and the boundaries it deliberately does not claim past.

PAGES 2–3

Implementation

Every feature, what was built, and where to verify it.

PAGE 4

Automated results

Lint, typecheck, 289 unit tests, the four-dimension eval, the build.

PAGE 5

The eval harness

Case by case: input, expectation, observed result.

PAGES 6–8

Edge cases & scope

Failure handling, live and manual verification, and the roadmap.

PAGE 9

What it does not claim

The boundaries this build draws deliberately, and why each one is stated rather than hidden.

Diagnosis and teaching

Routes are live paths in the deployed application; file paths are relative to the repository root.

FEATURE	WHAT WAS IMPLEMENTED	HOW IT CAN BE VERIFIED
Check My Work	Photo or typed capture, client-side compression, async pipeline returning an id immediately, status polled from real stage boundaries	Live at /scan; pipeline in lib/ai/pipeline/orchestrator.ts
Step verification	Adjacent-pair checking routed by shape to algebraic, quantitative or chemical verifiers; free-variable guard returns “could not verify”	npm test — math-verifier, multi-subject, solve-step
Divergence audit	Five-state classification per line, downstream propagation, independent-error detection, derived corrected line	npm run eval:deterministic; divergence & solution-audit tests
Misconception catalogue	17 coded entries across four subjects plus an explicit UNCLASSIFIED; signature detection marks a code proved vs matched	lib/diagnosis/misconceptions.ts; basis shown on every gap in /gaps
Grounded explanation	Local keyword retrieval over 58 curated chunks, filtered by kind; chunk ids recorded per call	lib/ai/rag/retrieve.ts; ids visible in /dev/observability
Computed diagrams	14 renderers selected deterministically; parameters computed from the student's own verified working, or a curated example on the explainer path only	components/visuals/; a test asserts a diagnosis context never renders a curated example
Ask a Concept	Intent detection and routing to one of 22 fixed slugs, lesson assembly, concept check built from catalogued distractors	Live at /learn; concept-explainer.test.ts
Solve With Me	Step-at-a-time guidance that names the next legal move without writing the line, checked by the same verifier	Live at /solve; guided-solve.test.ts
AI Coach	Retrieval-grounded, with the learner's real record in context and an explicit refusal when that context does not cover the question	Live at /coach

Mastery and platform

Every row names something that can be opened, run, or read — nothing here asks to be taken on trust.

FEATURE	WHAT WAS IMPLEMENTED	HOW IT CAN BE VERIFIED
Practice & teach-back	Problems generated for the specific misconception and solved independently before display ; teach-back scored against a rubric	<code>practice-generation.test.ts</code> asserts every generated problem is independently solvable
Exam Mode	Two-question minimum, reasoning weighted separately from correctness, a returning code decisive, uncertain a real verdict	<code>exam-verdict.test.ts</code> ; rules in <code>lib/exam/verdict.ts</code>
Mastery & memory	0–100 EMA ($\alpha = 0.35$), transfer at +12 against practice's +6; <code>LearningEvent</code> and <code>LearningMemory</code> written alongside	<code>prediction.test.ts</code> ; <code>lib/ai/pipeline/update-mastery.ts</code>
Roadmap & achievements	Ordering over 22 concepts and 29 prerequisite edges; five milestones with machine-checkable criteria	Live at <code>/roadmap</code> , <code>/achievements</code> ; criteria in <code>prisma/seed.ts</code>
Provider cascade	<code>cache</code> → Gemini → Groq → deterministic, with a per-error-class retry policy and offline substitutes for every stage after vision	<code>providers.test.ts</code> ; fallbacks in <code>lib/ai/fallback/</code>
Research sources	OpenAlex, Crossref and arXiv with two deterministic relevance gates; no prompt anywhere in the layer	<code>resources-registry.test.ts</code> pins the <i>silences</i> as well as the hits
Observability	Per-call logging of stage, provider, cache status, latency, error text and chunk ids, replayed as a trace	Live at <code>/dev/observability</code> ; <code>AiUsageLog</code> in the schema
Auth & isolation	Edge JWT verification, per-route session re-check, <code>userId</code> -scoped queries, 404 on another user's record	<code>src/middleware.ts</code> ; verified live — see page 7

What running the suite actually produces

`npm run verify` runs all four gates and **passes with no API keys configured at all** — which is the point: the load-bearing claims of this product do not depend on a model being reachable.

289

unit tests passing
across 18 files

45

eval assertions scored
0 failed, 2 skipped

100%

on all four measured
dimensions

0

lint warnings
and type errors

200

live deployment
responding

GATE BY GATE

COMMAND	RESULT
<code>npm run lint</code>	No ESLint warnings or errors
<code>npx tsc --noEmit</code>	Clean — strict, with <code>noUncheckedIndexedAccess</code>
<code>npm test</code>	18 files, 289 passed (289)
<code>npm run eval:deterministic</code>	45/45 assertions · 0 failed · 2 skipped
<code>npm run build</code>	Succeeds — exit 0, 87.3 kB shared JS

Two guards worth calling out. `navigation.test.ts` parses the tab bar's item list and the layout's route table *from source* and asserts they match both ways — this existed as a live bug, where a tab navigated to a page with no tab bar. And `resources-registry.test.ts` pins the *silences*: a paper on “distributive lattices” must be *rejected* for a distributive-property gap.

Tests that assert something does *not* happen are the ones that keep an honest product honest.

WHAT EACH TEST FILE PINS

FILE	GUARDS
<code>math-verifier</code>	Step-to-step validity
<code>divergence</code>	First-divergence location
<code>solution-audit</code>	Root error vs. carried consequence
<code>solve-step</code>	Derived corrected lines
<code>guided-solve</code>	Step-at-a-time solving
<code>practice-generation</code>	Generated problems are solvable
<code>prediction</code>	Prediction and mastery scoring
<code>exam-verdict</code>	Exam grading, and refusal to over-claim
<code>concept-explainer</code>	Intent routing and lesson assembly
<code>multi-subject</code>	Chemistry and biology verifiers
<code>providers</code>	Provider config, vision opt-in, model choice
<code>resources-registry</code>	Subject gating and the relevance gate
<code>navigation</code>	Tab bar and route table agreement
<code>spotify-pkce</code>	PKCE challenge construction

The core claim, measured case by case

Fifteen fixed worked solutions with known error locations, run through the *real* verification and divergence engine. It touches no model, no database and no network — so a regression in the deterministic core cannot hide behind a good day from an API.

What each case asserts: (1) the first divergence is found at the right step; (2) every later step is classified correctly as a downstream consequence or an independent error; and (3) where a corrected line is derived, solving it yields the same answer as solving the line it came from — so a “correction” can never quietly change the problem.

CASE	INPUT SHAPE	EXPECTED	OBSERVED	STATUS
correct-work-01	Valid working throughout	No divergence	No divergence	PASS
sign-error-01	Sign lost at step 2, carried on	Step 2 + downstream	Step 2, +2 downstream	PASS
wrong-final-answer-01	Error only at the last line	Step 3	Step 3	PASS
multiple-errors-01	Two unrelated mistakes	Step 2 + 1 independent	Step 2, +1 independent	PASS
negative-numbers-01	Correct work with negatives	No divergence	No divergence	PASS
negative-numbers-sign-flip-01	Sign flipped on a negative	Step 2	Step 2	PASS
distribution-correct-01	Brackets expanded correctly	No divergence	No divergence	PASS
distribution-partial-01	Minus reached the first term only	Step 2 + downstream	Step 2, +1 downstream	PASS
factoring-correct-01	Valid factoring	No divergence	No divergence	PASS
fractions-01	Correct work with fractions	No divergence	No divergence	PASS
fractions-common-denominator-error-01	Denominators mishandled	Step 2	Step 2	PASS
alternative-valid-approach-01	Right answer by an unusual route	No divergence	No divergence	PASS
word-problem-01	Prose framing, valid working	No divergence	No divergence	PASS

The behaviours that separate a careful system from a confident one

Each of these is a case where the correct behaviour is to do *less* — decline, stay silent, or say so. They are the hardest behaviours to build and the easiest to fake, and every one is enforced in code.

EDGE CASE	CORRECT BEHAVIOUR	WHERE IT IS ENFORCED
Three free variables <code>v = u + a*t</code>	Return "could not verify" , never a verdict. An earlier version accused students who were entirely correct	Free-variable guard in the algebraic verifier; covered by math-verifier
An unusual but valid method	No divergence reported	Eval case alternative-valid-approach-01
Lines that cannot be checked	If <i>every</i> line is unverifiable, fail with an honest message — never report "nothing wrong". If only some are, say exactly that: <i>"2 lines couldn't be checked, but everything we could verify holds"</i>	orchestrator.ts; analysis result assembly
Ambiguous handwriting	Pause at needs_confirmation and ask the student to check the reading before diagnosing	Confirm screen at /analysis/[id]/confirm
Vision unavailable	TYPE_INSTEAD: — the app offers typed working, which runs the identical pipeline with no AI	The one stage with no local substitute, stated as such
Both providers unavailable	Offline classifier, chunks surfaced directly, template practice — each reporting low confidence out loud	lib/ai/fallback/
One correct exam answer	Verdict uncertain — equally consistent with understanding and a lucky guess	exam-verdict.test.ts

What was checked by hand, and what came back

The automated gates cover the deterministic core. These are the checks that need a running deployment or a human in a browser — recorded here rather than implied.

DEPLOYMENT VERIFICATION — CHECKED LIVE

Application root	GET / → 200 , served from Vercel
Public API	GET /api/auth/me → 200 (allow-listed as public)
Auth gate, live	GET /home with no session → 307 redirect — route protection is live in production, not only in development

INTEGRATION VERIFICATION

Unit-tested	Provider selection, vision opt-in, subject gating, the relevance gates, PKCE construction, and every failure path
Verified by hand	The success paths of OpenAlex, Crossref, arXiv, YouTube, GitHub and Spotify — pinning them means CI flakiness or drifting fixtures
The guarantee	Remove any external service — or all of them — and the diagnosis still runs, the gap is still found, and practice still grades

THE MANUAL MATRIX — WALKED BY HAND BEFORE A RELEASE

Core path	New user · existing user · photo upload · typed working · diagnosis · first gap · reasoning replay · visual · voice · practice · transfer · teach-back · mastery update	Spotify	Not configured · configured but not connected · free tier · Premium · expired refresh token · no active device · consent denied · state mismatch
Degradation	No API keys · invalid keys · Gemini quota exceeded (Groq takes over) · both providers down (deterministic path) · slow API · retry · empty research · cached-only state	Layout	Mobile portrait · browser resize · tab bar present on every tab · scroll past the fold
Session	Login · logout · refresh · expired session · direct navigation to a protected route while signed out	Standards	No console errors · no dead buttons · no infinite spinners · no broken links · no fabricated success

What it covers today, and where it goes next

Everything described in this dossier is implemented and running. This page draws the line between what the system **proves** and what it extends into next.

PROVED TODAY — DECIDED BY CODE, MEASURED BY THE HARNESS

Step validity	Every transition checked by <code>mathjs</code> , routed per step-pair to the algebraic, quantitative or chemical verifier
First divergence	Located and classified across five states, with downstream consequences separated from independent errors
Corrected line	Derived from the student's own opening line, and asserted to preserve the solution
Misconception	18 catalogue codes, proved outright where the algebraic signature is unambiguous
Mastery & next step	EMA scoring capped until the concept transfers; eleven ordered rules choose what happens next

RUNNING TODAY — VERIFIED IN LIVE USE

Handwriting reading	Gemini vision reads photographed working across all four subjects, pausing to ask when a line is genuinely ambiguous
Concept routing	Questions routed to the seeded concept graph, with the model selecting from fixed slugs rather than inventing them
Grounded teaching	Explanations grounded in 58 curated chunks, with the chunk ids recorded per call and visible in the trace view
Live sources	OpenAlex, Crossref, arXiv, YouTube and GitHub returning real metadata behind two relevance gates
Full journey	Walked end to end before every release against the matrix on the previous page

WHERE IT EXTENDS — AND WHY THE EXTENSION IS CHEAP

Wider algebra

Quadratics and multi-variable rearrangement follow the same verifier contract — a domain module, not a redesign.

More subjects

A verifier, a routing rule, catalogue entries and seeded chunks. Diagnosis, teaching, practice and mastery come for free.

More providers

One interface inherits the cascade, caching, logging and fallback. No component holds provider-specific code.

Deeper corpus

Concepts, chunks and codes are data, not logic — growing the curriculum never touches the engine.

What this build does not claim

A system that reports its own boundaries can be trusted on the things it does assert. Each line below is a place where the honest answer was to record less than we could have displayed.

HINTS ARE NOT YET COUNTED

Independence is derived from the attempt index — a row count, and therefore a fact. A first-attempt success after reading a hint still scores as unaided, because no surface currently records that the hint was read.

Why it is stated rather than papered over. The parameter exists and is honoured everywhere downstream; what is missing is the input, not the mechanism. Inferring it would mean guessing at a number the product then shows a student as if it were measured.

SIGNATURES ARE DEEPEST IN ALGEBRA

Linear algebra has the richest set of provable signatures. Chemistry and physics have fewer; biology has none, and its diagnoses are matched against the catalogue by the model rather than proved from the numbers.

The distinction is stored, not hidden. Every diagnosis carries a basis of proved or matched, and the two are never merged — so a model-chosen label can always be told apart from an algebraic proof.

THE EVAL IS A REGRESSION HARNESS

Forty-five assertions over fixed fixtures whose expectations were written by hand, as pedagogical claims independent of the implementation. It runs in milliseconds with no model, no database and no network.

What that entitles us to say. That the engine still behaves exactly as specified, and that a regression cannot slip through unnoticed. Not that the numbers generalise to arbitrary unseen student work — a claim that size would need a dataset this project does not have.

HANDWRITING IS VERIFIED BY USE, NOT BY FIXTURE

Two eval cases need an image fixture and none exists, so the harness reports them as SKIP and leaves them out of the denominator rather than quietly counting them as passes.

The reading stage is exercised live across all four subjects before every release, and the pipeline pauses to ask the student when a line is genuinely ambiguous — which is the correct behaviour for a system that must never accuse someone of a mistake they did not make.

None of these weakens the central claim. Locating the first divergence, separating carried error from fresh error, proving the misconception where the algebra allows it, and choosing what happens next are all decided by deterministic code and measured every run. The limits above sit at the edges of that core — and naming them is what makes the numbers in the middle worth reading.